Programs for a Frozen Machine: The Description Length of Language and Task in Optimized Prefixes

Seminar report, Mechanistically Interpreting Multilingual Language Models (WT 2025/26)

Georgii Sirotenko

October 2026

1 Abstract

A frozen language model is a fixed machine; a soft prefix that makes it perform a task is a program for that machine, and the shortest such program is a model-relative description length of the task. This report measures that length for a frozen Qwen2.5-1.5B base model over a grid of three tasks (translation from English, factual recall, sentence continuation) and three output languages (Russian, German, English), in two units: prefix slots and variational bits. Four results. (i) Slots are not a description length: translation saturates at one slot, a single trained slot beats the natural-language prompt everywhere, and a conditioning string is worth less than one slot. (ii) In bits, a translation program costs about 500 bits at 10% excess loss, a Russian recall program five times that, and English recall most of all; continuation has nothing to compress. (iii) In bits, neither task nor language is the cheap modifier of the other, and sharing does not compress. Under the preregistered protocol one centre per task with language deviations coded all eight cells for 37% fewer bits than one centre per language with task deviations; rerun under the corrected protocol, the two nestings cost the same within 25% at matched hyper-parameters, a language given a task costs what a task given a language costs (about 200 bits for translation, 3,200 for recall), and the task-centred hierarchy costs 1.5 times eight independent programs, parity on translation and 1.9 times on recall, because one centre is priced by its dearest member and one β must serve all of them. The asymmetry is in the mechanism, not in the bits: recall programs are language-specific, do not compose zero-shot and are read deep in the network, whereas translation programs are read almost entirely in the first block; inside a hierarchical prefix the language deviation is read again at blocks 16–17, which the task centre is not. (iv) The variational objective selects compact programs but the standard optimiser does not find them: a conservative learning rate and checkpoint selection on the full objective lower the measured frontier by 20–70% with fewer runs, which turns the open problem of Shaw et al. (2026) into a practical recipe at this scale.

2 Question and framing

For a frozen model M, task t and output language l, let $K_M(t, l)$ be the shortest soft prefix that performs t in l at a chosen distortion. The project asks how $K_M(t, l)$ is distributed between language and task, whether the two parts are separable, how far apart the programs for different cells lie, and where in depth the model reads them. All lengths are relative to one model, tokenizer and format; the claims are orderings, differences and ratios within that model.

Two units are used throughout. **Slots** are the number k of prefix positions, a structural unit of attention. **Bits** are the description length of the prefix under a prior: the prefix is a diagonal-Gaussian posterior and its KL divergence to a prior fitted to the token-embedding matrix is its code length under the bits-back argument (Hinton and van Camp 1993; Blier and Ollivier 2018). This is the measurement used by Shaw et al. (2026), who treat prompt tokens as a program tape and the Transformer as the interpreter. Counting tokens, the convention of prompt tuning (Lester et al. 2021), is a capacity measure, and Section 4.1 shows why it fails

as a description length.

Distortion is the teacher-forced code length L of the target in bits per target token, normalised per cell as $\hat{D} = (L - L_{\text{floor}})/(L_{\text{none}} - L_{\text{floor}})$, with L_{none} the no-prefix loss and L_{floor} the better of a per-cell LoRA fine-tune and the best prefix run. The operating points $\hat{D} \leq 0.25$ and $\hat{D} \leq 0.10$ were preregistered from a dev pilot; $\hat{D} \leq 0.05$ is reported as exploratory.

3 Setup

Model and data. `Qwen/Qwen2.5-1.5B` base, frozen, bfloat16 on an A100. Translation and continuation from FLORES+ (dev split for training and selection, devtest for reporting); recall from mLAMA (Kassner et al. 2021), with an English cloze prefix and the object rendered in the target language, split by uuid. Translation into English is excluded, so the grid has eight cells.

Designs. *Design A*: $k \in \{1, 2, 4, 8, 16, 32\}$ free slot vectors, two seeds, 1500 steps, no penalty; repeated with the target language given, with the task given, and the natural-language prompt with no prefix. *Flat variational*: $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$, one KL to the global prior. *Hierarchical*: one centre per task with a deviation per language, and the reverse nesting, $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$, deviation prior $\mathcal{N}(\mu_{\text{centre}}, \tau^2\sigma_0^2 I)$ with $\tau = 1$ in units of the prior σ_0. *Composition*: tied task parts and language parts, six parts for eight cells, with five hold-out sets. *Distance*: warm-start prequential learning cost from every cell's $k = 4$ prefix toward every cell over 512 examples. *Regime 2*: per-example input vectors, $m \in \{1, 2, 4, 8\}$, with and without the task prefix. *Mechanism*: the prefix positions' hidden states from a trained prefix patched into one block's attention input of a run with a random prefix, for every block.

Two variational protocols. The preregistered protocol (`flat`) used learning rate 0.05, the checkpoint with the lowest dev distortion at the posterior mean, and posterior σ initialised at five times the prior. Its histories (Section 4.8) showed that this protocol overshoots the rate and selects on the wrong statistic. A corrected protocol (`flat2`) uses learning rate 0.01, dev distortion scored on posterior samples, the checkpoint with the lowest dev objective (distortion plus β times the rate), and σ initialised at the prior. `flat2` was run on the five measurable cells at $k \in \{2, 4\}$, $\beta \in \{1, 3\}$ and is the headline where it exists; `flat` is reported alongside. The hierarchical design was run under both: the preregistered selection rule at $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$ (with the corrected σ initialisation), and the corrected protocol on the `flat2` grid, $k \in \{2, 4\}$, $\beta \in \{1, 3\}$. Both nestings were rerun under the corrected protocol, which is the headline for every hierarchical comparison; the preregistered hierarchical runs are reported for the record.

Inventory. 273 Design A, baseline and floor rows; 72 flat and 20 flat2 rows; 144 hierarchical rows (54 joint runs) under the preregistered rule and 64 (24 joint runs) under the corrected protocol; 72 composition rows; 72 distance rows; 32 regime 2 rows; 1,120 mechanism rows. About 34 A100 hours. Every number below is recomputed by `notebooks/01_figures.ipynb` from the CSVs.

4 Results

4.1 Slots are not a description length (H1, slot unit; H2, slot unit)

Translation reaches $\hat{D} \leq 0.10$ with one slot in every language and conditioning, and its curve is flat from there. Recall keeps improving to $k = 32$ on Russian and German (English is not monotone) with the budget binding (a 3000-step run on recall-ru at $k = 8$ moves $\hat{D}$ from 0.043 to 0.032 and is still falling). Continuation is noise: its floor lies within 0.10–0.16 bits per token of the no-prefix loss, so the denominator of $\hat{D}$ is below the seed variation, and continuation is excluded from every headline claim. Seed gaps are below 0.011 on translation, 0.042 on recall-ru and recall-de, and 0.20 on recall-en, the one cell where the slot curve is not reliable.

Table 1 gives k^*, the smallest k reaching each level.

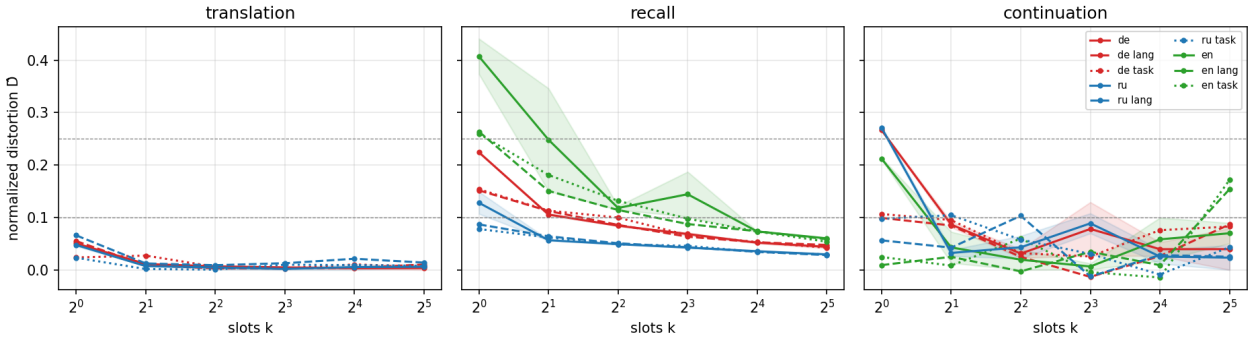


Figure 1: $\hat{D}$ against the number of slots k, Design A, seed means with the seed range shaded; dashed lines are the operating points; line style is the conditioning format.

cell	$k^*(0.25)$	$k^*(0.10)$	$k^*(0.05)$, expl.
translation-ru	1	1	1
translation-de	1	1	2
recall-ru	1	2	4
recall-de	1	4	32
recall-en	2	16	not reached

Two observations make the slot count unusable as a description length. First, the natural-language prompt, scored as a context with no prefix, is far worse than one trained slot: on recall it leaves $\hat{D}$ at 0.91–0.96, one slot at 0.13–0.41; on translation it leaves 0.12–0.20, one slot 0.05. One vector of 1,536 numbers carries more than an instruction, so "one token" is not a unit of information. Second, conditioning saves little on the slot axis: giving the language or the task changes $k^*(0.10)$ in none of the translation cells and in two recall cells (recall-ru 2 to 1, recall-en 16 to 8, for either conditioning). Δk_{lang} and Δk_{task} averaged over the measurable cells are both 1.8 slots, and 8 of those 9 slots come from recall-en. The slot axis cannot see the difference between telling the model the language and telling it the task; the bit axis can (Section 4.3).

4.2 Description length in bits (H1, bit unit)

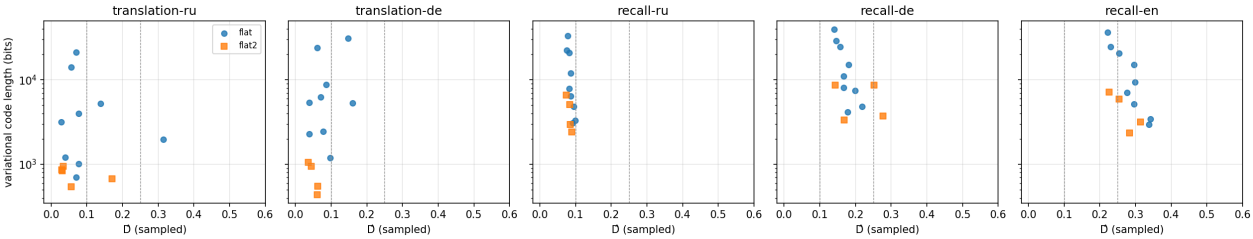


Figure 2: Rate against distortion per cell, every variational run; circles are the preregistered protocol, squares the corrected one. Distortion is scored on posterior samples.

Table 2 is the headline: the smallest code length among runs reaching each operating point, with the preregistered protocol beside the corrected one.

cell	bits at $\hat{D} \leq 0.10$ (flat2)	bits at $\hat{D} \leq 0.25$ (flat2)	flat2 best $\hat{D}$	flat at $\hat{D} \leq 0.25$
translation-de	436	436	0.036	1,194
translation-ru	542	542	0.030	699
recall-ru	2,419	2,419	0.075	3,101
recall-de	not reached	3,343	0.143	4,168
recall-en	not reached	7,155	0.226	24,686

A translation program costs about 500 bits. A Russian recall program costs five times that; German and En-

recall is the dearest program, not the cheapest, in slots ($k^*(0.10) = 16$) and in bits, and the result survives restricting all three recall test sets to the 4,212 uuids they share ($\hat{D}$ at $k = 4$: ru 0.049, de 0.079, en 0.122). The English LoRA floor is also the worst of the three (2.58 bits per token against 1.10 for Russian), so what is dear in English is not the language but the facts that mLAMA asks about in English. H1's language part is not supported; the exposure proxy from the companion project was not available and the comparison rests on orderings.

4.3 Neither nesting is cheaper, and sharing does not compress (H2, bit unit)

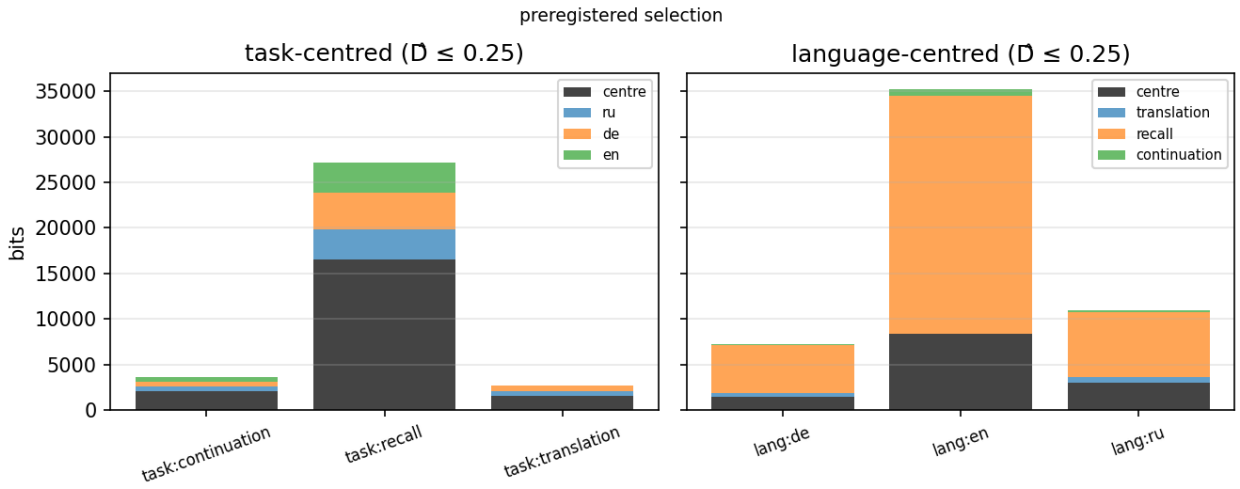


Figure 3: Code length at $\hat{D} \leq 0.25$ under the two hierarchical nestings: the centre is paid once per group, each deviation once per member.

The hierarchical design codes a group of cells as one centre plus one deviation per member. Table 3 compares the codes for all eight cells at $\hat{D} \leq 0.25$, the only level every cell reaches. Under the preregistered selection rule all three codes reach it. Under the corrected protocol, on the flat2 grid, the task-centred hierarchy and the independent programs reach it (flat2 on the five measurable cells, flat on continuation, which flat2 did not run), but the language-centred hierarchy does not: its best recall-en distortion is 0.28, so it codes the Russian and German cells (8,900 bits) and not the grid.

code	protocol	bits
one centre per task, language deviations	preregistered selection	33,500
eight independent flat programs	preregistered	37,600
one centre per language, task deviations	preregistered selection	53,500
one centre per task, language deviations	corrected	26,700
one centre per language, task deviations	corrected	not reached (ru + de: 8,900)
eight independent flat programs	corrected	17,600

Under the preregistered rule grouping by task costs 37% less than grouping by language, and the task-centred code is 11% cheaper than eight independent programs. Neither margin survives the corrected protocol, with both sides selected on the objective and scored on samples; both were selection artefacts of the kind described in Section 4.8. Sharing does not compress: the task-centred hierarchy costs 26,700

bits against 17,600 for independent programs, a factor of 1.5. Table 4 gives the comparison per group. The task-centred hierarchy is at parity on translation and costs 1.9 times the independent programs on recall; the language-centred hierarchy is at parity on Russian and German (1.04 and 1.08) and fails on English. At matched (k, β) a hierarchy costs 0.94–1.5 times the sum of the flat2 runs with the same hyper-parameters, at similar or worse distortion, so the shared centre buys nothing; the frontier comparison then adds the cost of one β for every member. On recall the only task-centred run in which all three languages reach 0.25 is $\beta = 1$, whose centre alone (14,600 bits) costs more than the three independent programs together. Nor is either nesting cheaper than the other: at matched (k, β) the language-centred code costs 0.88–1.24 times the task-centred one over all eight cells, at equal or worse distortion (Table 5). The 37% gap of the preregistered runs came from language-centred recall deviations of 5,300–26,200 bits, which the corrected protocol brings down to 3,200–3,400 for Russian and German.

group, $\hat{D} \leq 0.25$	hierarchy, corrected (k, β)	independent, corrected	ratio
translation (ru, de)	1,029 (4, 3)	978	1.05
recall (ru, de, en)	24,972 (4, 1)	12,917	1.93
continuation (ru, de, en; unmeasurable)	667 (4, 3)	3,724 (flat)	0.18
Russian (translation, recall, continuation)	4,353 (2, 3)	4,204	1.04
German (translation, recall, continuation)	4,560 (2, 3)	4,227	1.08
English (recall, continuation)	not reached	9,188	—

Table 4. Task-centred (top) and language-centred (bottom) hierarchies against independent programs per group under the corrected protocol; the independent continuation values are from the preregistered flat runs.

(k, β)	task-centred, bits (worst $\hat{D}$)	language-centred, bits (worst $\hat{D}$)	ratio
(2, 1)	24,591 (0.28)	30,448 (0.59)	1.24
(2, 3)	12,693 (0.56)	11,182 (0.48)	0.88
(4, 1)	28,659 (0.23)	30,532 (0.28)	1.07
(4, 3)	11,180 (0.28)	13,396 (0.37)	1.20

Table 5. The two nestings at matched hyper-parameters under the corrected protocol: three centres plus eight deviations either way, summed over all eight cells, with the worst member distortion.

The per-member costs are symmetric under the corrected protocol. Given the translation centre (604 bits), a target language costs 225 (ru) or 200 (de) bits, 0.41–0.46 of a standalone program; given a language centre (830–900 bits), translation costs 201 (ru) or 257 (de). Given the recall centre (14,600 bits), a language costs 3,200–3,900 bits, 1.2–1.3 times a standalone Russian or German program; given a language centre, recall costs 3,200 (ru) or 3,400 (de), and for English no run reaches 0.25 (2,600 bits at 0.37, 6,500 at 0.28). Continuation given a language centre costs 60 bits. Under the preregistered rule the same quantities were 485 and 662 for a language given translation, 3,300–4,000 for a language given recall, and 440–630, 5,300–26,200 and 110–740 for translation, recall and continuation given a language; the asymmetry was in the optimiser. Recall is the program that cannot be shared with anything: it costs about 3,200 bits as a deviation from either kind of centre, and whichever centre must serve it (the recall centre, or the English centre) is the expensive one.

For translation alone the hierarchy is at best at parity: centre plus two deviations costs 1,029 bits at $\hat{D} \leq 0.10$ under the corrected protocol against 978 for two independent programs (2,733 against 1,893 under the

preregistered rule). The centre is cheaper than either specialised program and each deviation is less than half of one, but centre plus deviation is not. H2 in bits therefore fails in both directions: the task given the language costs 0.9–1.0 (ru, de) times the language given the task on recall and 0.9–1.3 on translation, where the preregistered rule had given 1.3–7.8 on recall. This agrees with the slot axis, where $\Delta k_{\text{lang}} = \Delta k_{\text{task}}$.

4.4 Composition (H3)

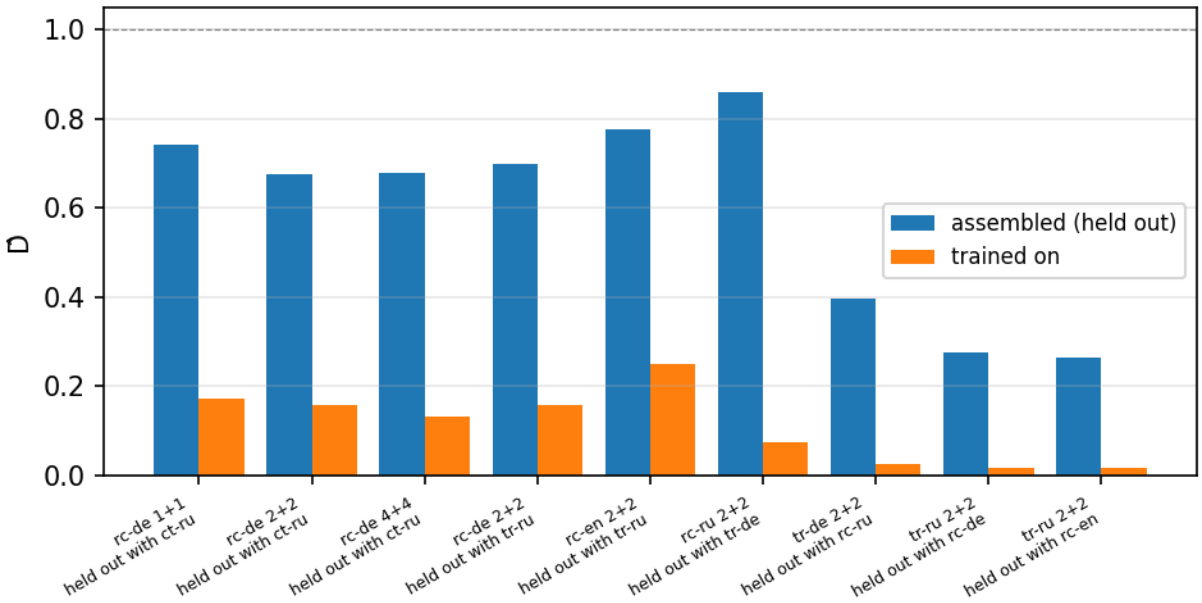


Figure 4: Assembled (held-out) against trained-on $\hat{D}$ of tied task + language parts.

Six tied parts trained on every cell match Design A at the same total slots on translation ($\hat{D}$ 0.017–0.027 against 0.005–0.007 at $2 + 2$) and cost about twice its distortion on recall (0.08–0.25 against 0.05–0.12). Assembled zero-shot for a held-out cell, the parts close 60–73% of the gap between no prefix and the floor on translation (three hold-outs over two cells, $\hat{D}$ 0.27–0.40) and 14–32% on recall (six hold-outs over three cells, $\hat{D}$ 0.68–0.86). Task and language parts compose partially for translation and barely for recall, consistent with the per-member costs above. The Chinese comparison in the preregistered test was not run; the resource-level part of H3 is untested.

4.5 Distance between programs (H4)

Warm-starting from a prefix of the same task costs 0.85 of training from scratch; from the same language or an unrelated cell, 0.95. The silhouette of the task partition on the symmetrised distance is 0.004 and that of the language partition -0.09: the direction is H4's, the structure is absent. The matrix is one seed, seven pairs per group, and several warm starts cost more than scratch, so it measures optimisation noise as much as distance. The zero-shot block is the strong signal: a prefix applied to any other cell is roughly as bad as, or worse than, no prefix ($\hat{D}$ 0.7–1.4, median 0.97, up to 2.5), and a recall prefix applied to continuation is catastrophic ($\hat{D}$ 5–25). The recall program overrides the model's default behaviour rather than adding to it.

4.6 Where the program is read (H5)

Patching all blocks reproduces the donor exactly. Patching block 0 alone recovers 70–87% of the effect for translation and 74–94% for continuation, but only 36–58% for recall, which has a secondary peak at block



Figure 5: Left: symmetrised warm-start learning cost divided by the from-scratch cost. Right: $\widehat{D}$ of a trained prefix applied unchanged to another cell.



Figure 6: Fraction of the prefix's effect recovered by patching its positions into one block's attention input, mean over cells per task. Fractions are undefined where the donor is not at least 0.1 bits per token better than the recipient, which excludes the deviation probes on continuation and on translation-ru.

17 and a smaller one at blocks 10–11. Single-block fractions summed over blocks come to 2.6 for translation and 3.0 for continuation, so their reading is redundant, but only 1.3 for recall, which is close to additive. A translation program is copied into the token stream in the first block and not needed afterwards; a recall program is read in several places, deep into the network.

The preregistered comparison, the block of maximal recovery for the language deviation against the task centre, gives block 0 for every probe on every cell, so the peak does not separate them. The profiles do. The task centre is read early: on recall its early mass (blocks 0–3) is 1.0 and its late mass (blocks 14–27) 0.5. The language deviation on top of it is read early and again deep: on recall early 1.6, late 1.9, with block 17 alone recovering 0.94 of the deviation's effect; on translation-de early 1.2, late 2.1, with blocks 16 and 22 recovering 0.99 and 0.85. The task deviation under a language centre behaves like the task centre (recall: early 1.0, late 0.5). So there is a depth split by prefix part, in the opposite direction to H5's expectation: the task part is read early, the language part early and late. The same reading was offered in the previous draft with the two deviation probes transposed; the figure's third panel is the language deviation. One detail: the recall centre without its English deviation is worse than a random prefix on English, so the recall centre is a non-English program.

4.7 Input compressibility (H6, exploratory)

With the $k = 4$ task prefix, one per-example vector replaces the input to within 10% of the real-input loss on every cell but translation-ru, which needs two; without the prefix, translation and continuation need two. There is no rate term on the vectors, so this is a slot count and is reported as such. The per-sentence against per-token comparison in H6 was not computed. One oddity for the record: on recall the bare model scores the target better with no input (3.1 bits per token) than with the real query (7.2), which is consistent with confident wrong answers once a subject is named.

4.8 Optimisation: the objective is right, the search is hard

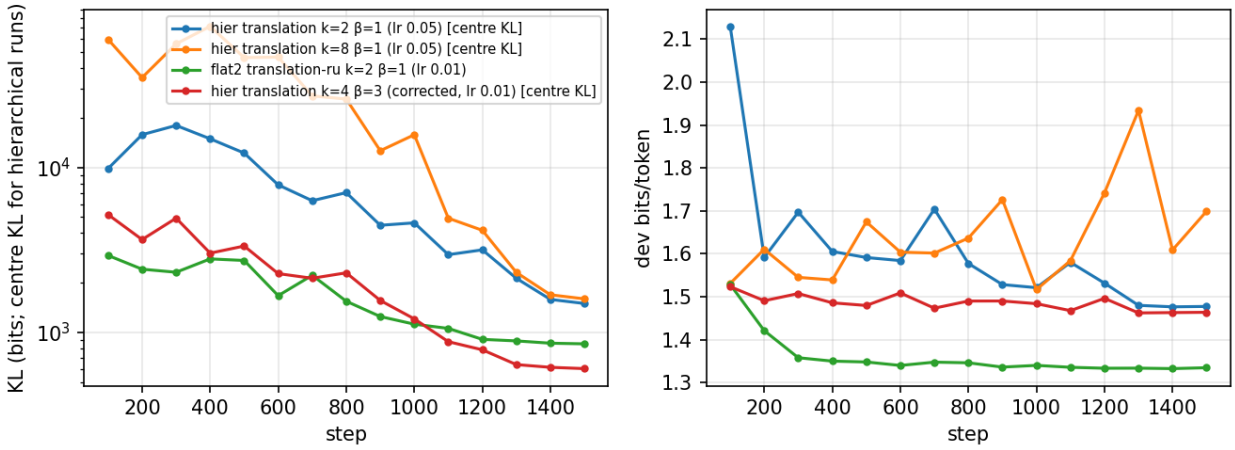


Figure 7: KL and dev loss against training step for two hierarchical runs under the preregistered protocol, one flat2 run and one hierarchical run under the corrected protocol.

The saved histories of the 27 task-grouped hierarchical runs show three things. With learning rate 0.05, two prior standard deviations per Adam step, the centre KL overshoots early, peaking between steps 100 and 800 (median 200–400), by an amount that grows with k: a median peak of 26,000 bits at $k = 2$ (range 6,000–182,000) and 144,000 at $k = 8$ (28,000–549,000), before the cosine schedule lets the KL term pull the means back. In the plotted translation runs the final centre rate is the same at $k = 8$ as at $k = 2$ (1,600 against 1,500 bits), but the posterior-mean distortion worsens late at $k = 8$, so the distortion-only checkpoint rule selects a step where the rate is ten times the final one; across all runs the selected rate exceeds the final one

by a median factor of 1.1–1.2 and up to 20. And the posterior mean is not the coded object: on translation at $\beta = 3$ the mean scores $\hat{D}$ 0.35–0.56 in three of four runs while samples score 0.06–0.17, because the model was trained on samples (on recall the two agree).

The 12 task-grouped hierarchical runs under the corrected protocol show none of them: the centre KL peaks at step 150–200 at a median 3,600 ($k = 2$) to 5,700 bits ($k = 4$), 7–12 times lower, then falls monotonically, and the selected checkpoint is within 2% of the final rate. The corrected protocol removes all three effects and lowers the measured frontier on every cell, by 22% (translation-ru, recall-ru; 29% for translation-ru at the 0.05 level), 58–63% (translation-de), 20% (recall-de) and 71% (recall-en), with four runs per cell instead of nine, and the rates become monotone in β. Shaw et al. (2026) report that on a parity task the same objective reaches 870 bits from a hand-compiled initialisation but 2.8 million bits from random initialisation, and leave the optimiser as the open problem. At this scale the gap is closed mostly by selecting on the objective and stepping at the scale of the prior.

5 Preregistered comparisons

	comparison	outcome
H1	cost ordering by language exposure and by task difficulty	task ordering supported (recall $\gg$ translation); language ordering reversed on recall (en dearest), robust to common items; exposure proxy unavailable
H2	Δk_{lang} vs Δk_{task}; bit savings under Design B	equal on the slot axis (1.8 slots each, 8 of 9 from one cell); in bits, equal under the corrected protocol (task given language 0.9–1.0× language given task on recall, 0.9–1.3× on translation); the preregistered 1.3–7.8× asymmetry on recall does not replicate
H3	composition cost at $\hat{D} = 0.25$; Design B minus Design A gap	translation composes partially (60–73% of the gap zero-shot), recall barely (14–32%); a shared centre saves no bits: under a common protocol the task-centred hierarchy costs 1.5× eight independent programs (parity on translation, 1.9× on recall) and the two nestings cost the same within 25%; Chinese not run
H4	silhouette of task vs language clustering	0.004 vs -0.09: task direction, no structure, not significant (7 pairs per group, one seed); same-task warm start 0.85 vs 0.95
H5	block of maximal recovery, language deviation vs task centre	block 0 for both; but the language deviation is read again at blocks 16–17 and the task centre is not: a split by part, opposite in direction to H5

	comparison	outcome
H6	m^* per sentence and per token across languages	one vector suffices with the task prefix; per-token variant not computed; no rate term

6 Deviations and limitations

Three deviations from the preregistration are recorded in the project ledger. The floor is the better of LoRA and the best prefix, because prefixes beat LoRA on five cells. Continuation is excluded from headline claims as unmeasurable. The variational protocol was corrected after the preregistered runs, for the reasons in Section 4.8, and the corrected runs are the headline; both are reported. Two seeds were preregistered where the proposal had planned three, and the recall budget binds at 1500 steps. Both nestings were rerun under the corrected protocol on the flat2 grid ($k \in \{2, 4\}$, $\beta \in \{1, 3\}$, $\tau = 1$), so the hierarchical comparisons are like for like but rest on four runs per group and one τ. Under the corrected protocol every rate, flat or hierarchical, is still falling at the end of the 1500-step budget, so all are upper bounds. One corrected run (English centre, $k = 2$, $\beta = 1$) selected a transient dip in the dev loss at step 500; selection on the objective is not immune to dev noise. The distance matrix is one seed. The mechanism probes patch attention inputs at the prefix positions; a residual-stream patch would answer a slightly different question. The τ of the hierarchical design was first run in absolute embedding units, which made the deviation prior forty times wider than the global prior; those runs are kept as a control and not used.

7 What a research version would do

The same code runs the scale axis (Qwen2.5 0.5B to 7B), twenty or more FLORES languages stratified by resource, further tasks, and discrete prefixes. Three changes follow from this report. Select every variational checkpoint on the objective and score distortion on samples. Replace the learning rate by a step size in units of the prior σ. Adopt the per-column Gaussian-mixture prior of Shaw et al. (2026) as a second rate, and charge $\log_2 |k\text{-grid}|$ bits for the choice of k. The hypotheses worth the compute are the ones this report could not settle: the exposure ordering across many languages, the composition cost against resource level, and whether the depth at which a program is read changes with scale.

8 References

- Blier, L. and Ollivier, Y. (2018). The Description Length of Deep Learning Models. NeurIPS.
- Dumas, C. et al. (2025). Separating Tongue from Thought. ACL.
- Hendel, R., Geva, M. and Globerson, A. (2023). In-Context Learning Creates Task Vectors. Findings of EMNLP.
- Hinton, G. E. and van Camp, D. (1993). Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights. COLT.
- Kassner, N., Dufter, P. and Schütze, H. (2021). Multilingual LAMA. EACL.
- Khashabi, D. et al. (2022). Prompt Waywardness. NAACL.
- Lester, B., Al-Rfou, R. and Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP.
- Li, X. L. and Liang, P. (2021). Prefix-Tuning. ACL.
- Shaw, P., Cohan, J., Eisenstein, J. and Toutanova, K. (2026). Bridging Kolmogorov Complexity and Deep Learning. ICLR. arXiv:2509.22445.
- Todd, E. et al. (2024). Function Vectors in Large Language Models. ICLR.
- Voita, E. and Titov, I. (2020). Information-Theoretic Probing with Minimum Description Length. EMNLP.
- Vu, T. et al. (2022). SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. ACL.

9 Appendix: reproduction

Repository `sirotenkogeorgii/prefix-programs`, branch `prefixlab`. `results_sync/` holds every result table, per-example bits, training histories and the prefix checkpoints. `notebooks/01_colab_runner.ipynb` reproduces the runs on an A100 (about 32 hours); `notebooks/01_figures.ipynb` regenerates every figure and table here from the CSVs and `prereg.json` in under a minute; `notebooks/02_live_demo.ipynb` trains one prefix live.